

Unit 4

Transport Layer and End-to-End Protocols: Transport layer services including UDP and TCP, flow control, congestion control, TCP congestion avoidance algorithms, session management (establishment and synchronization), and presentation layer functions such as data formatting, encryption, and decryption.

TRANSPORT LAYER

Introduction

- The Transport Layer is Layer 4 of the OSI model.
- It provides end-to-end (process-to-process) communication between applications running on different devices.
- It acts as a bridge between:
 - Application Layer
 - Network Layer

Also called the End-to-End Layer because it ensures communication from source process to destination process.

Key Responsibilities

1. End-to-End Delivery

- Ensures data reaches the correct destination process
- Uses port numbers for identification.

2. Segmentation and Reassembly

- Breaks large data into smaller units called segments.
- At receiver side, segments are reassembled in correct order.

3. Process-to-Process Communication

- Enables multiple applications to communicate simultaneously.
- Example:
 - Browser → Port 80

- Email → Port 25

4. Multiplexing and Demultiplexing

Allows multiple applications to communicate at the same time.

Multiplexing: Data from multiple applications is combined at sender side.

Demultiplexing: Data is delivered to the correct application at receiver side.

Example: You can use browser, email, and chat apps at the same time, and each receives its own data correctly.

5. Error Control

- Detects errors using checksums.
- Lost or corrupted data is **retransmitted**.

6. Flow Control

- This controls the **rate of data transmission** between sender and receiver.
- Prevents receiver from being overwhelmed.
- Example: If receiver slow then sender slows down.

7. Congestion Control

This manages traffic in the network.

Problem: Too much data in the network can cause congestion.

Solution: Transport layer reduces the sending rate.

Example: Internet becomes slow during peak hours due to high traffic

8. Reliability

The transport layer ensures:

- No data loss
- No duplication
- Correct order of data

How it works:

- Uses acknowledgements (ACK)

- Lost data is retransmitted

Example:

While downloading a file, if some part is missing, it is automatically sent again.

9. Connection-Oriented Service (TCP)

In this service, a **connection is established before sending data**.

How it works:

- First, a connection is created between sender and receiver
- Then data is transferred
- Finally, the connection is closed

Key points:

- Reliable communication
- No data loss
- Data arrives in correct order

Example:

When you open a website, your browser first establishes a connection with the server. After that, the data is transferred safely.

10. Connectionless Service (UDP)

In this service, **no connection is established** before sending data.

How it works:

- Data is sent directly
- No confirmation if it is received or not

Key points:

- Faster than TCP
- Not reliable
- No guarantee of delivery

Example:

In video streaming or online gaming, some data loss is acceptable, so speed is preferred over reliability.

Working of Transport Layer

At sender:

1. Receives data from Application Layer
2. Divides data into segments/datagrams
3. Adds header (port no., sequence no., etc.)
4. Sends to Network Layer
5. At receiver:
 - Removes header
 - Reassembles data
 - Sends to correct application

Transport Layer Protocols

1. TCP (Transmission Control Protocol)
2. UDP (User Datagram Protocol)
3. SCTP (Stream Control Transmission Protocol)

Advantages of Transport Layer

- Reliable communication
- Error detection & correction
- Supports multiple applications
- Efficient data transfer

Disadvantages

- Extra overhead (headers, control)
- TCP slower due to reliability mechanisms

Transmission Control Protocol

Transmission Control Protocol (TCP) is the most widely used transport layer protocol. It works with IP and provides a reliable, end-to-end communication service between processes.

Services Provided by TCP

1. Process-to-Process Communication

TCP enables communication between applications running on different systems.

- Uses 16-bit port numbers
- Identifies both sender and receiver processes

2. Stream-Oriented Service

TCP treats data as a **continuous stream of bytes**.

- Data is divided into **segments**
- Each segment is sent with a header and then encapsulated into IP packets

3. Full-Duplex Communication

TCP allows simultaneous data transfer in both directions.

- Both sender and receiver can send data at the same time

4. Connection-Oriented Service

TCP establishes a connection before data transmission.

It consists of three phases:

- Connection Establishment
- Data Transfer
- Connection Termination

5. Reliability

TCP ensures reliable data delivery by using:

- **Checksums** → error detection
- **Acknowledgements (ACKs)** → confirm receipt
- **Retransmission** → resend lost data
- **Sequence numbers** → maintain order
- **Congestion control** → manage network load

6. Multiplexing and Demultiplexing

TCP supports multiple connections over a single physical network.

- Multiplexing (sender side): Combines data from multiple applications
- Demultiplexing (receiver side): Delivers data to the correct application

Components of TCP Header

TCP headers mainly consist of three components:

1. Byte Number

- Every byte in a TCP connection is assigned a unique number.
- Numbering starts from an arbitrary initial value chosen at the start of the connection.

2. Sequence Number

- Represents the byte number of the first byte in the current segment.
- Helps the receiver correctly reassemble data, even if segments arrive out of order.

3. Acknowledgement Number

- Indicates the next expected byte number from the sender.
- Confirms that all bytes up to this number have been successfully received.

3-Way Handshake (TCP Connection Establishment)

Used to establish a reliable connection. Ensures both sides are ready before communication.

Steps:

1. SYN(Synchronize)

Client sends a TCP segment with SYN=1, including its ISN (Initial Sequence Number).

Marks the request to initiate a connection

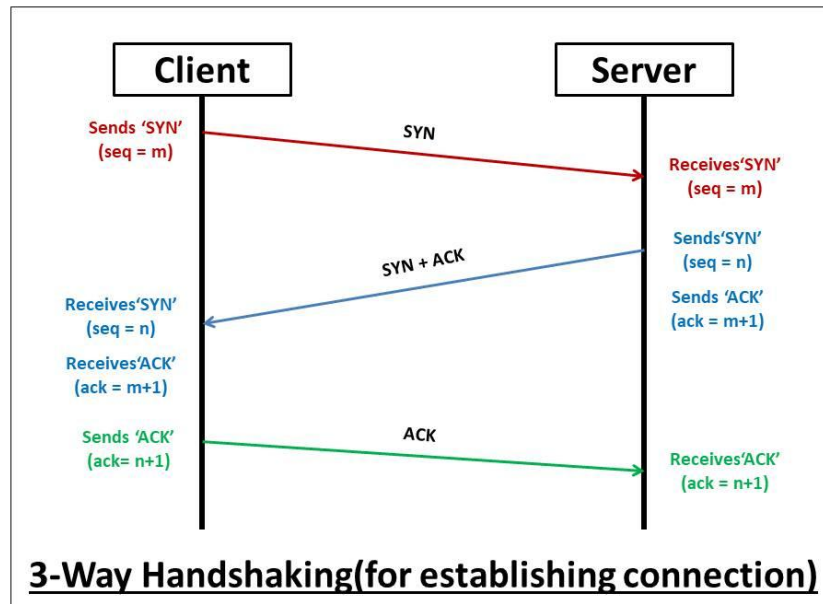
- Request to start connection

2. SYN-ACK(Synchronize-Acknowledge)

- Server → Client
- Server replies with SYN=1 & ACK=1, containing its own ISN and ACK = client_ISN + 1.
- Confirms receipt of client's SYN and initiates its own sync
- Acknowledges request

3. ACK(Acknowledge)

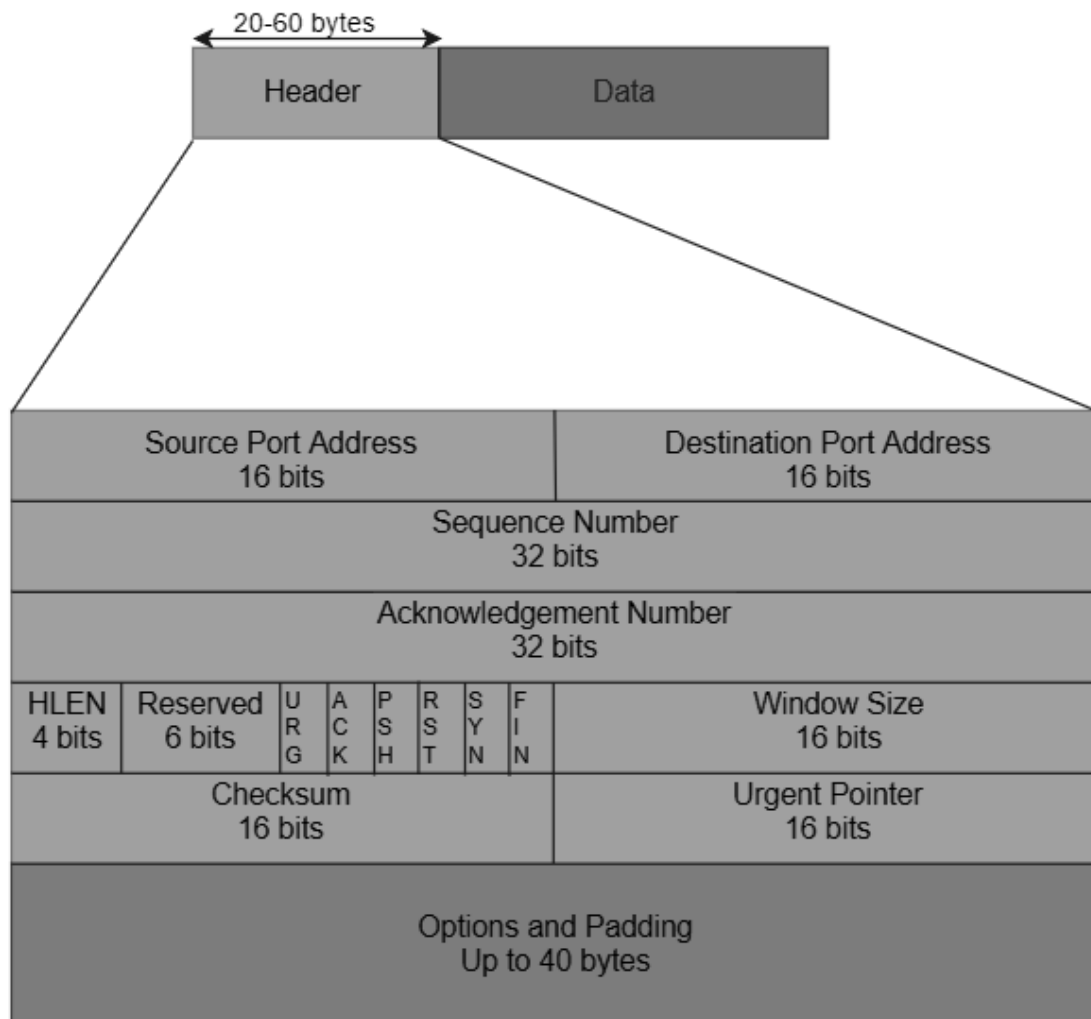
- Client → Server
- Client sends an ACK=1 segment with ACK = server_ISN + 1.
- Completes synchronization; connection enters ESTABLISHED state



TCP Segment structure

A TCP segment consists of:

- Header (20–60 bytes)
- Data (payload)



1. Source Port (16 bits): Identifies sending application

2. Destination Port (16 bits): Identifies receiving application

3. Sequence Number (32 bits):

- Byte number of first data byte in segment
- Helps in reordering data

4. Acknowledgement Number (32 bits)

- Next expected byte
- Acts as confirmation of received data

5. Header Length (HLEN)

- Size of TCP header
- Measured in **4-byte units**
- Range:

- Minimum → 20 bytes
- Maximum → 60 bytes

6. Control Flags (6 bits)

These control communication:

- **URG** → urgent data present
- **ACK** → acknowledgement valid
- **PSH** → push data immediately
- **RST** → reset connection
- **SYN** → establish connection
- **FIN** → terminate connection

7. Window Size

- Used for **flow control**
- Tells how much data receiver can accept

8. Checksum

- Used for **error detection**
- Ensures data integrity

9. Urgent Pointer

- Points to **urgent data**
- Used only when URG flag is set

UDP (User Datagram Protocol)

- UDP (User Datagram Protocol) is a transport layer protocol.
- It provides fast, connectionless communication between processes.
- Works with IP

It is called connectionless because:

- No connection setup
- No handshake
- Data is sent directly

Characteristics of UDP

- Connectionless protocol
- No acknowledgement (ACK)
- No retransmission
- No ordering of data
- Low overhead
- Very fast

Working of UDP

1. Application sends data to UDP
2. UDP creates a datagram (packet)
3. Adds header:
 - Source port
 - Destination port
 - Length
 - Checksum
4. Datagram is passed to IP layer
5. IP sends it to destination
6. Receiver delivers data to correct application

No checking whether data reached or not

UDP Services

1. Process-to-Process Communication

- Uses **port numbers (16-bit)**
- Identifies sending and receiving applications

2. Connectionless Service

- No connection establishment
- No connection termination

3. Fast Data Transmission

- No delay due to acknowledgements
- Suitable for real-time applications

4. Multiplexing and Demultiplexing

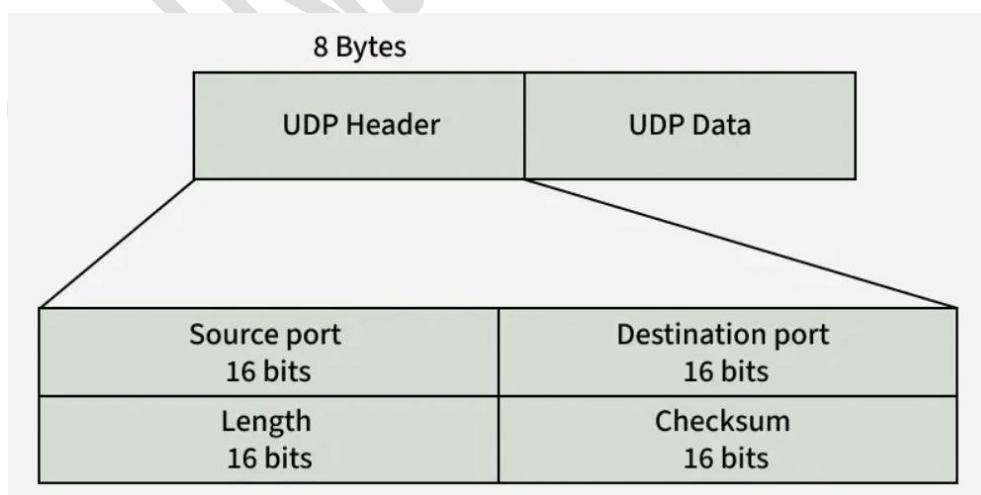
- Multiple applications can use UDP
- Data delivered using port numbers

5. Minimal Error Checking

- Uses **checksum** for basic error detection
- No correction mechanism

UDP Header Format

UDP header is **fixed size = 8 bytes**



Fields:

1. Source Port (16 bits): Sender application port

2. Destination Port (16 bits): Receiver application port

3. Length (16 bits): Total size of UDP header + data

4. Checksum (16 bits)

- Error detection
- Optional in IPv4, mandatory in IPv6

After header, **data (payload)** is present

Applications

UDP is used where **speed is more important than reliability** because Small delay is acceptable, but **speed is critical** :

- DNS (Domain Name System)
- Video streaming
- Online gaming
- VoIP (voice calls)
- DHCP

Advantages of UDP

- Very fast
- Low overhead
- Simple protocol
- Suitable for real-time communication

Disadvantages of UDP

- No reliability
- No error recovery
- No flow control
- No congestion control
- Data may be lost or arrive out of order

COMPARISON

TCP	UDP
Connection-oriented	Connectionless
Reliable	Unreliable
Slow	Fast
Uses acknowledgement	No acknowledgement
Maintains data order	Does not maintain order
Supports retransmission	No retransmission
Flow control present	No flow control
Congestion control present	No congestion control
Header size: 20–60 bytes	Header size: 8 bytes
Stream-based communication	Message-based communication
Used in HTTP, FTP, Email	Used in DNS, Gaming, Streaming

Flow Control in TCP

Flow control in TCP is a mechanism that ensures the sender transmits data at a rate that the receiver can handle, preventing buffer overflow and data loss.

Need for Flow Control

In a network, the sender and receiver may operate at different speeds:

- Sender may be fast
- Receiver may be slow or have limited buffer space

If the sender sends too much data:

- Receiver buffer may overflow
- Data may be lost

Therefore, flow control is required to **regulate the data transmission rate**.

TCP uses a **window-based mechanism** to control the flow of data.

- The sender is allowed to send a **limited amount of data**
- This limit is called the **window size**

- The window size is determined by the **receiver**

Key Concepts of TCP Flow Control

- Receiver Window (rwnd): A field in the TCP header where the receiver informs the sender of available buffer space.
- Sliding Window Protocol: The sender maintains a window of packets it can send without receiving an ACK. As ACKs arrive, the window slides forward, enabling further transmission.
- Buffer Management: When the receiver's buffer fills up, it advertises a smaller rwnd. If the buffer is full, it sends $\text{rwnd} = 0$, forcing the sender to pause.
- Window Scaling: To support modern high-speed networks, TCP uses a window scale option to expand the 16-bit window field from 64 KB up to 1 GB.

Receiver-Side Control

Flow control is mainly controlled by the **receiver**.

Advertised Window (rwnd):

- Receiver sends its available buffer size to the sender
- This value is called the **receiver window (rwnd)**

Working:

- Sender adjusts its sending rate based on rwnd
- Ensures receiver is not overloaded

Zero Window Condition

- If receiver buffer becomes full $\rightarrow \text{rwnd} = 0$
- Sender stops sending data
- When buffer becomes free $\rightarrow$ receiver sends updated window size
- Sender resumes transmission

Sender-Side Perspective

- Sender maintains a sending window
- It can send data up to the allowed window size
- It waits for ACKs before sending more data

Key Features

- Prevents receiver overload
- Ensures efficient data transmission
- Allows multiple segments to be sent at once
- Dynamically adjusts transmission rate

CONGESTION CONTROL

Congestion control is a technique used in computer networks to control the amount of data entering the network so that the network does not become overloaded.

It ensures that the network operates efficiently by preventing too many packets from being sent at once, which could otherwise lead to delays and data loss.

Example

- Suppose a router can handle 500 packets per second
- But 900 packets per second are sent to it
- The extra packets are dropped, causing congestion

Congestion control reduces the sending rate so that packets match the network capacity.

Why Congestion Control is Needed

- To avoid packet loss
- To reduce delay in data transmission
- To improve network performance
- To ensure fair usage of bandwidth among users

Congestion control works by either

- preventing congestion before it happens, or
- detecting congestion and then reducing the sending rate

Flow Control	Congestion Control
Prevents receiver overload	Prevents network overload
Works between sender and receiver	Works across the entire network
Focuses on receiver capacity	Focuses on network capacity
End-to-end control mechanism	Network-wide control mechanism
Uses techniques like sliding window	Uses techniques like TCP congestion algorithms
Ensures receiver is not overwhelmed	Ensures network is not congested

Open Loop Congestion Control

- Open loop congestion control tries to prevent congestion before it happens
- It does not use feedback from the network
- All decisions are fixed in advance

Techniques

1. Retransmission Policy

- Controls when and how retransmission should happen
- Prevents unnecessary duplicate packets which increase load

Example

- Sender waits for proper timeout instead of retransmitting immediately

2. Window Policy

- Controls number of packets sent before receiving acknowledgement
- Proper window size avoids flooding the network

Example

- Selective Repeat retransmits only lost packets, reducing traffic

3. Acknowledgement Policy

- Reduces number of acknowledgements in the network
- Helps lower overall traffic

Example

- One ACK is sent for multiple packets instead of each packet

4. Discard Policy

- Drops less important packets during congestion
- Helps reduce load quickly

Example

- Router drops multimedia packets instead of important data

5. Admission Control

- Controls entry of new traffic into network
- Prevents overload

Example

- Network blocks new connections when already busy

Closed Loop Congestion Control

What it means

- Closed loop congestion control detects congestion and reacts
- It uses feedback from the network
- Sender adjusts its speed dynamically

Techniques

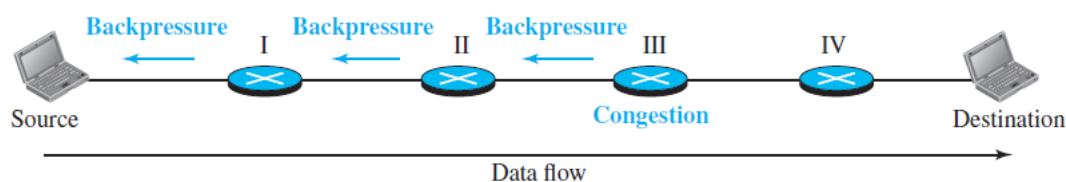
1. Backpressure

- A congested node sends signal to previous node to slow down
- Control moves step by step toward sender

Example

- Router B tells Router A to stop sending data temporarily

Figure 18.14 Backpressure method for alleviating congestion



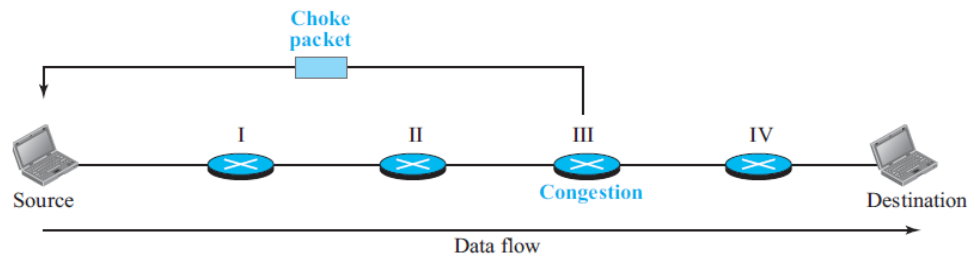
2. Choke Packet

- Router sends a control packet directly to sender
- Sender reduces its transmission rate

Example

- Router sends “slow down” message to source

Figure 18.15 Choke packet



3. Implicit Signaling

- Sender detects congestion by observing delay or packet loss
- No direct message is sent

Example

- Increase in delay indicates congestion, so sender slows down

4. Explicit Signaling

- Network directly informs sender about congestion
- More accurate and faster than implicit method

Example

- Router sets congestion bit in packet header

TCP Congestion Control

What is TCP Congestion Control

- TCP congestion control is used to control the number of segments sent into the network
- It prevents congestion in intermediate routers, not just at sender or receiver
- TCP adjusts its sending rate based on network condition

Why Congestion Control is Needed in TCP

- Flow control (rwnd) only prevents receiver overload
- But congestion can still happen inside the network (routers)
- Routers receive data from multiple senders → buffers overflow → packets lost

Important idea

- More packet loss → more retransmissions → more congestion → network collapse

Congestion Window (cwnd)

- TCP uses a variable called **cwnd (congestion window)**
- It limits how many segments can be sent into the network

Key Point

- Actual sending window = minimum of rwnd and cwnd

So

- rwnd controls receiver side
- cwnd controls network side

Example

- rwnd = 10 segments
- cwnd = 6 segments

Actual window = 6 segments

Sender can send only 6 segments

Congestion Detection

TCP detects congestion using ACK behavior

Two signals

1. Timeout

- If ACK is not received within time
- TCP assumes **severe congestion**

Example

- Packet sent but no ACK arrives → timeout → packet assumed lost

2. Three Duplicate ACKs

- Receiving 3 duplicate ACKs means a packet is missing
- Indicates **mild congestion**

Example

- ACK for packet 2 keeps repeating → packet 3 is lost

TCP Congestion Control Policies

TCP uses three main algorithms

- Slow Start
- Congestion Avoidance
- Fast Recovery

1. Slow Start (Exponential Increase)

- At the beginning, TCP has **no idea about network capacity**.
- So it starts very cautiously and then increases speed rapidly to find the limit.
- Start with small congestion window
- Increase rapidly to utilize bandwidth

Working

- cwnd starts from 1 MSS
- For each ACK received
 $\text{cwnd} = \text{cwnd} + 1$
- This results in exponential growth per RTT

Growth Pattern

- $1 \rightarrow 2 \rightarrow 4 \rightarrow 8 \rightarrow 16$

Why It Becomes Exponential

RTT 1

- Send 1 packet → get 1 ACK → $\text{cwnd} = 2$

RTT 2

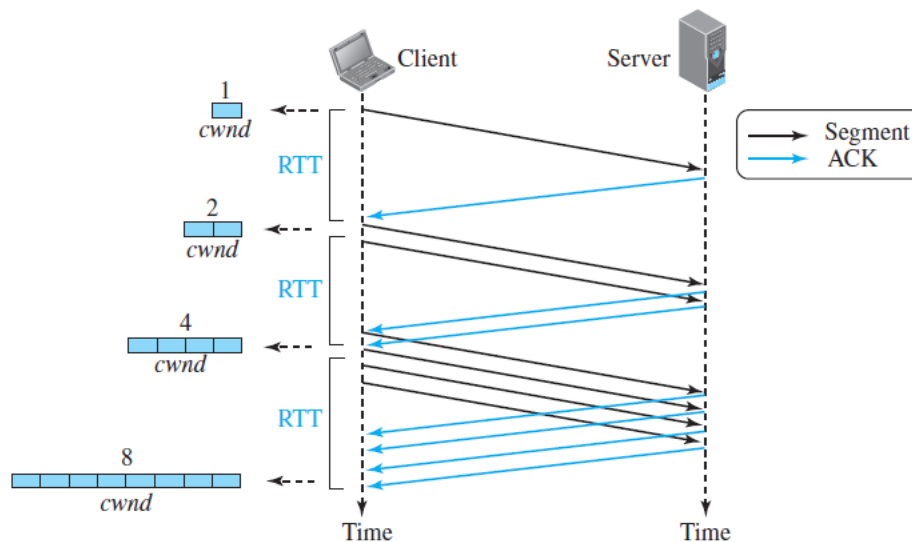
- Send 2 packets → get 2 ACKs → $\text{cwnd} = 4$

RTT 3

- Send 4 packets → get 4 ACKs → $cwnd = 8$

So growth is doubling every RTT

Figure 24.29 *Slow start, exponential increase*



Thus, In the slow-start algorithm, the size of the congestion window increases exponentially until it reaches a threshold.

2. Congestion Avoidance

Congestion avoidance is a phase in TCP congestion control where the sender **increases the congestion window slowly** to avoid overloading the network.

Purpose

- To prevent congestion before it occurs
- To maintain stable data transmission
- To avoid sudden packet loss due to high traffic

When It Starts

- It begins when **cwnd reaches the slow start threshold (ssthresh)**
- After slow start phase ends

Working

- The growth of congestion window becomes **linear**

- Instead of doubling, cwnd increases slowly

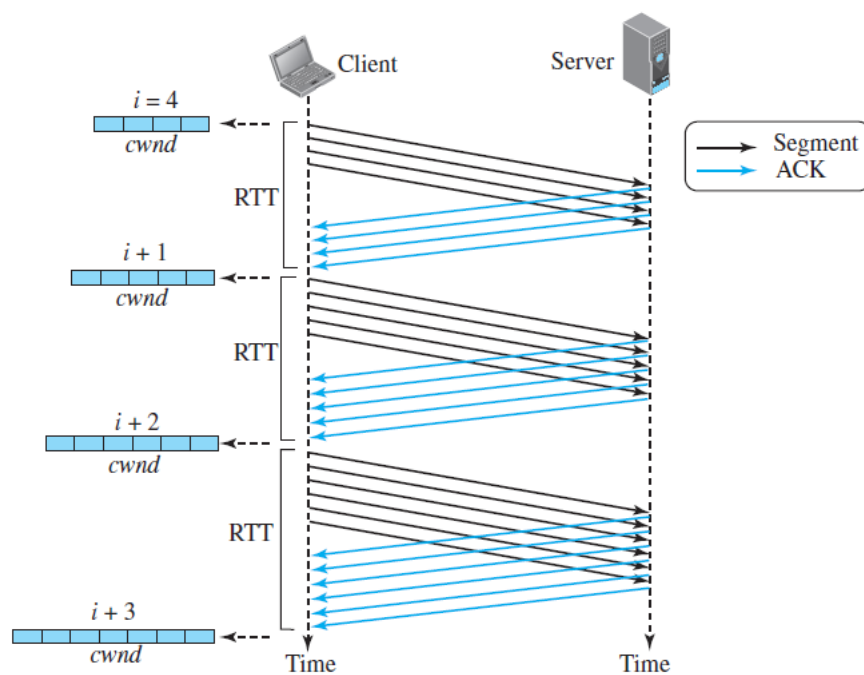
Rule

- cwnd increases by **1 MSS per RTT**

Growth Pattern

- cwnd = 16 → 17 → 18 → 19 → 20

Figure 24.30 Congestion avoidance, additive increase



Example:

Assume cwnd = 16 and ssthresh = 16

Step 1: Sender sends 16 segments

Step 2: All ACKs received successfully

Step 3: cwnd increases to 17

Step 4: Next RTT → cwnd becomes 18

Key Characteristics

- Growth is linear
- Safer than slow start

- Avoids sudden congestion
- Utilizes network bandwidth efficiently

Behaviour on Packet Loss

Case 1: Timeout (Severe Congestion)

- cwnd reset to 1
- Slow start begins again

Case 2: 3 Duplicate ACKs (Mild Congestion)

- cwnd reduced to half
- Fast recovery phase starts

3. Fast Recovery

Fast recovery is a TCP congestion control technique used to handle mild congestion by reducing the congestion window and continuing transmission without going back to slow start.

Purpose

- To avoid restarting transmission from the beginning
- To maintain better performance after packet loss
- To handle congestion efficiently without drastic slowdown

When It Starts

- When the sender receives 3 duplicate ACKs
- Indicates that one packet is lost but others are arriving
- So congestion is not severe

Working

Step 1: Sender receives 3 duplicate ACKs

Step 2: Lost packet is immediately retransmitted (Fast Retransmit)

Step 3: Congestion window is reduced to half

Step 4: Transmission continues with **linear growth** (like congestion avoidance)

Example

Assume

- $cwnd = 20$

Packet loss detected (via 3 duplicate ACKs)

Then

- $cwnd$ becomes 10
- Now grows slowly:
 $10 \rightarrow 11 \rightarrow 12 \rightarrow 13$

Key Characteristics

- Does not reset $cwnd$ to 1
- Reduces window size to control congestion
- Maintains continuous data flow
- Faster recovery compared to slow start

Why Fast Recovery is Needed

- Not all losses mean heavy congestion
- If only one packet is lost, restarting from $cwnd = 1$ is inefficient
- Fast recovery keeps network utilization high

Thus,

- Fast recovery assumes network is still working
- So it reduces speed but **does not stop aggressively**

Slow Start	Congestion Avoidance	Fast Recovery
Initial phase of TCP transmission	Second phase after threshold	Third phase after packet loss
Increases $cwnd$ rapidly	Increases $cwnd$ slowly	Reduces $cwnd$ and then increases slowly
Growth is exponential	Growth is linear	Linear growth after reduction
$cwnd$ starts from 1 MSS	Starts when $cwnd \geq ssthresh$	Starts after 3 duplicate ACKs

Used to quickly utilize bandwidth	Used to avoid congestion	Used to recover from mild congestion
Stops at ssthresh or packet loss	Continues until packet loss	Continues after reducing cwnd
Aggressive approach	Conservative approach	Balanced approach

Silly Window Syndrome (SWS)

Silly Window Syndrome is a problem in TCP where very small segments are transmitted repeatedly, leading to inefficient use of network bandwidth.

Concept

- TCP is designed to send data in large segments for efficiency
- In SWS, instead of sending large chunks, **small amounts of data are sent frequently**
- This increases overhead because each segment carries headers along with very little data

Causes of Silly Window Syndrome

SWS can occur due to **two main reasons**:

1. Sender-Side Problem

- The sender application generates data in **small pieces**
- TCP sends each small piece immediately without waiting

Example

- Data generated: 1 byte at a time
- Sender sends:
 - 1 byte → packet 1
 - 1 byte → packet 2
 - 1 byte → packet 3

Result

- Large number of tiny packets are sent

- Network efficiency decreases

2. Receiver-Side Problem

- The receiver processes data slowly
- It frees buffer space in **small amounts**
- It advertises very small window size (rwnd)

Example

- Receiver buffer = 100 bytes
- It frees only 1 byte at a time

Receiver advertises:

- rwnd = 1, then 2, then 3

Sender sends very small segments repeatedly

Result

- Many small packets are transmitted
- Bandwidth is wasted

Problems Caused by SWS

- Large number of small packets
- High protocol overhead (headers > data)
- Poor utilization of bandwidth
- Reduced network performance

Solutions to Silly Window Syndrome

1. Nagle's Algorithm (Sender-Side Solution)

- Sender does not send small packets immediately
 - It waits until:
 - Enough data is collected
- OR

- Previous packet is acknowledged

Example

- Instead of sending 1 byte multiple times
- Sender collects data and sends one large segment

2. Clark's Solution (Receiver-Side Solution)

- Receiver does not advertise small window sizes
- It waits until sufficient buffer space is available

Example

- Instead of advertising $rwnd = 1$
- Receiver waits and advertises larger window like $rwnd = 50$

Key Idea

- Avoid sending very small packets
- Send data in larger chunks for better efficiency

SOLVED NUMERICALS ON TCP

Q1. A TCP connection has a congestion window of 40 KB when a timeout occurs. The RTT is 50 ms and MSS = 2 KB. Find the time required for the congestion window to reach 40 KB again.

Answer:

After timeout:

cwnd = 1 MSS = 2 KB

ssthresh = $40/2 = 20$ KB

Slow Start:

$2 \rightarrow 4 \rightarrow 8 \rightarrow 16 \rightarrow 20$

= 4 RTTs

Congestion Avoidance:

Increase from 20 to 40 KB

Step = 2 KB per RTT

RTTs = $(40 - 20)/2 = 10$

Total RTTs = $4 + 10 = 14$

Time = $14 \times 50 \text{ ms} = 700 \text{ ms}$

Q2. A file of 7000 bytes is transmitted over TCP. The first byte number is 5000. The data is sent in 5 segments of sizes: 1000, 1000, 1000, 1000, and 3000 bytes. Find the sequence numbers of each segment.

Answer:

Segment	Size	Sequence Number
1	1000	5000
2	1000	6000
3	1000	7000
4	1000	8000
5	3000	9000

Q3. In TCP congestion control, initial cwnd = 1 MSS and ssthresh = 16 MSS. Assuming no packet loss, show the growth of cwnd for 6 transmission rounds.

Answer:

Round	cwnd
1	1
2	2
3	4
4	8
5	16
6	17

Q4. A TCP connection uses slow start with initial cwnd = 1 MSS. Find the number of RTTs required to send 1023 segments.

Answer:

Data sent doubles every RTT:

$$1 + 2 + 4 + 8 + 16 + 32 + 64 + 128 + 256 + 512 = 1023$$

RTTs required = 10

Q5. In a TCP connection, initial cwnd = 2 MSS and ssthresh = 20 MSS. A timeout occurs at the 6th transmission round. Find the congestion window at the end of the 10th round.

Answer:

Before timeout:

- Round 1 → 2
- Round 2 → 4
- Round 3 → 8
- Round 4 → 16
- Round 5 → 20
- Round 6 → Timeout

After timeout:

- ssthresh = $20/2 = 10$
- cwnd = 1

After timeout growth:

- Round 7 $\rightarrow$ 1
- Round 8 $\rightarrow$ 2
- Round 9 $\rightarrow$ 4
- Round 10 $\rightarrow$ 8

Final cwnd = 8 MSS

Q6. A TCP connection has RTT = 80 ms and window size = 16 KB. Calculate the throughput.

Answer:

Throughput = Window / RTT
 $= 16 \text{ KB} / 0.08 \text{ sec}$
 $= 200 \text{ KB/sec}$

Q7. In TCP, cwnd = 24 MSS and a triple duplicate ACK occurs. Find the new cwnd and ssthresh.

Answer:

ssthresh = $24/2 = 12 \text{ MSS}$
cwnd = 12 MSS

Q8. A TCP connection has initial cwnd = 1 MSS and ssthresh = 32 MSS. RTT = 40 ms. Find the time required to reach ssthresh.

Answer:

Growth:
 $1 \rightarrow 2 \rightarrow 4 \rightarrow 8 \rightarrow 16 \rightarrow 32$

RTTs = 5

Time = $5 \times 40 \text{ ms} = 200 \text{ ms}$

Q9. A TCP connection has cwnd = 32 MSS and ssthresh = 20 MSS.

A triple duplicate ACK occurs, followed by normal transmission for 3 RTTs, and then a timeout occurs. Find cwnd after the timeout and after 2 more RTTs.

Answer:

After triple duplicate ACK:

- ssthresh = $32/2 = 16$
- cwnd = 16

Next 3 RTTs (Congestion Avoidance):

- RTT1 $\rightarrow$ 17

- $RTT_2 \rightarrow 18$
- $RTT_3 \rightarrow 19$

Now timeout occurs:

- $ssthresh = 19/2 = 9.5 \approx 9 \text{ MSS}$
- $cwnd = 1 \text{ MSS}$

Next 2 RTTs (Slow Start):

- $RTT_1 \rightarrow 2$
- $RTT_2 \rightarrow 4$

Final $cwnd = 4 \text{ MSS}$

Q10. A TCP sender starts with $cwnd = 1 \text{ MSS}$ and $ssthresh = 8 \text{ MSS}$. After reaching $ssthresh$, a triple duplicate ACK occurs at $cwnd = 12 \text{ MSS}$. Find $cwnd$ for next 4 RTTs.

Answer:

Initial growth:

$1 \rightarrow 2 \rightarrow 4 \rightarrow 8$ (Slow Start)

Then:

$9 \rightarrow 10 \rightarrow 11 \rightarrow 12$ (Congestion Avoidance)

At $cwnd = 12 \rightarrow$ triple duplicate ACK:

- $ssthresh = 12/2 = 6$
- $cwnd = 6$

Next RTTs:

- $RTT_1 \rightarrow 7$
- $RTT_2 \rightarrow 8$
- $RTT_3 \rightarrow 9$
- $RTT_4 \rightarrow 10$

Final $cwnd = 10 \text{ MSS}$

SESSION LAYER

- 5th layer of OSI model
- Responsible for **establishing, managing and terminating sessions**
- Controls communication between applications
- **Acts as a conversation manager**

Key Responsibilities

1. Session Establishment

- Initiates connection between two devices
- Performs authentication
- Sets communication parameters

2. Session Maintenance

- Keeps session active
- Ensures smooth data exchange
- Maintains continuous communication

3. Session Termination

- Ends session after data transfer completes
- Ensures graceful closing (no data loss)

4. Dialog Control & Management

- Controls who can send data and when
- Communication modes:
 - Simplex – one-way
 - Half-duplex – one at a time
 - Full-duplex – both simultaneously

5. Synchronization (Checkpointing)

Data Transfer is in form of Checkpoints at intervals.

[Checkpoint 1] ---- [Checkpoint 2] ---- [Checkpoint 3]

If failure occurs → Resume from last checkpoint

- Inserts checkpoints in data stream
- Helps in error recovery
- Avoids restarting from beginning

6. Token Management

- Uses token mechanism
- Only device with token can transmit
- Prevents data collision

7. Recovery and Resynchronization

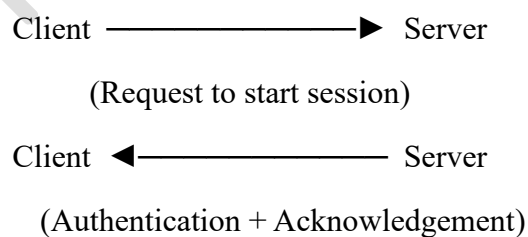
- Detects session failure
- Restores communication using checkpoints
- Avoids full retransmission

8. Session Control

- Maintains session ID and state
- Tracks active sessions
- Handles multiple connections

Working of Session Layer

1. Session Setup



- Communication begins with session establishment
- Client sends a request to initiate session
- Server performs authentication (if required)
- Parameters like mode (half/full duplex) are decided
- A session ID is created

2. Data Exchange Control

Client —▶ Server

Server —▶ Client

(turn-based or simultaneous communication)

- Session layer controls who sends data and when
- Implements dialog control
- Supports:
 - Half-duplex → one at a time
 - Full-duplex → both simultaneously
- May use token management for control

3. Synchronization (Checkpoints)

Data Flow:

[Start] —▶ [Checkpoint 1] —▶ [Checkpoint 2] —▶ [Checkpoint 3]

- Session layer inserts checkpoints in long data transfer
- Each checkpoint stores progress of transmission
- Helps in efficient recovery

4. Error Recovery

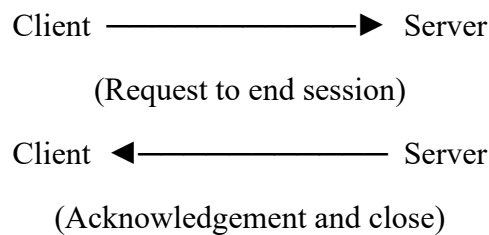
Failure occurs after Checkpoint 2

Resume from:

[Checkpoint 2] —▶ [Checkpoint 3]

- If failure occurs, session layer detects interruption
- Instead of restarting, it resumes from last checkpoint
- Saves time and bandwidth
- Ensures reliable communication session

5. Session Termination



- After data transfer, session is gracefully terminated
- Both sides release resources
- Ensures no data loss and proper closure

Protocols

1. PPTP (Point-to-Point Tunneling Protocol)

- Used to create VPN (Virtual Private Network)
- Establishes a secure session (tunnel) between client and server
- Data is encapsulated and transmitted securely over internet
- Helps in remote access to private networks

2. RPC (Remote Procedure Call)

- Allows a program to execute a function on another system
- Makes remote communication look like a local function call
- Session layer manages the request-response session
- Used in client-server applications

3. NetBIOS (Network Basic Input/Output System)

- Provides session-level communication in LAN
- Allows devices to establish and manage connections
- Supports:
 - Session establishment
 - Data transfer
 - Session termination
- Common in Windows-based networks

4. PAP (Password Authentication Protocol)

- Used for user authentication
- Sends username and password to server
- Server verifies and establishes session if valid
- Simple but less secure (no encryption)

5. RTP (Real-Time Control Protocol)

RTP → Carries media (audio/video)

RTCP → Sends feedback (quality, delay, loss)

- Works along with RTP (Real-Time Protocol)
- Does not carry data, instead sends control information
- Provides feedback like:
 - Packet loss
 - Delay
 - Jitter
- Helps in maintaining quality of multimedia sessions

Devices

- **Firewalls** – monitor sessions
- **Proxy Servers** – manage sessions
- **Session Border Controllers** – VoIP sessions
- **Application Servers** – maintain user sessions

Real-Life Examples

- **Video Calls** – start, maintain and end communication
- **File Download** – resumes using checkpoints
- **Login Systems** – maintain session until logout

PRESENTATION LAYER

- 6th layer of OSI model
- **Also called Translation Layer / Syntax Layer**
- Responsible for data representation and formatting
- Ensures that data sent by sender is understandable by receiver
- **Performs encryption, decryption, compression and translation**

Key Responsibilities

- Data translation (format conversion)
- Data encryption and decryption (security)
- Data compression and decompression (efficiency)
- Maintains syntax and structure of data

1. Data Translation (Format Conversion)

- Converts data from **sender format to a common format**
- At receiver side, converts it back to **receiver's format**
- Handles differences in:
 - Character encoding
 - Data formats
- Example:
 - ASCII ↔ EBCDIC
 - JSON ↔ XML

2. Data Encryption and Decryption

- Provides security during data transmission
- Converts readable data into unreadable (cipher text)
- Only authorized receiver can decrypt data
- Used in:
 - Secure websites (HTTPS)

- Online banking
- Example: SSL/TLS protocols

3. Data Compression and Decompression

- Reduces **size of data before transmission**
- Improves:
 - Transmission speed
 - Bandwidth utilization
- Types:
 - **Lossless compression**: no data loss (text files)
 - **Lossy compression** : slight data loss (images, videos)

4. Syntax and Semantics Management

- Maintains syntax (structure/format of data)
- Ensures semantics (meaning of data) is preserved
- Example:
 - Ensuring numbers, dates, formats are correctly interpreted
 - Converting structured data formats

5. Transfer Syntax Negotiation

- Sender and receiver agree on:
 - Data format
 - Encoding technique
 - Compression method
- Ensures both sides interpret data correctly
- Avoids miscommunication between systems

6. Interoperability

- Allows **different systems to communicate**

- Handles differences in:
 - Hardware architectures
 - Operating systems
 - Data representation formats
- Makes communication possible between **heterogeneous systems**

Protocols and Standards

1. SSL/TLS (Secure Sockets Layer / Transport Layer Security)

- Used for secure data transmission over network
- Provides:
 - Encryption (data becomes unreadable)
 - Authentication (verifies server/client)
 - Data integrity (no tampering)
- Works in HTTPS (secure websites)
- Example: Online banking, login pages
- Ensures confidential communication

2. JPEG, MPEG, GIF (Multimedia Formats)

Image → JPEG / GIF

Video → MPEG

- Define standard formats for multimedia data
- Used for encoding and decoding images/videos
- JPEG (Joint Photographic Experts Group)
 - Used for images/photos
 - Uses lossy compression (reduces size, slight quality loss)
- MPEG (Moving Picture Experts Group)
 - Used for video and audio
 - Compresses large video files for streaming
- GIF (Graphics Interchange Format)

- Used for simple images and animations
- Supports limited colors and looping animation
- Presentation layer ensures these formats are properly interpreted

3. ASCII and EBCDIC (Character Encoding)

'A' → ASCII → 65

'A' → EBCDIC → Different code

- Used to represent characters in binary form
- Each character is assigned a numeric code
- ASCII (American Standard Code for Information Interchange)
 - Most widely used encoding system
 - Used in modern computers and networks
- EBCDIC (Extended Binary Coded Decimal Interchange Code)
 - Used mainly in IBM systems
- Presentation layer converts data between different encoding formats

4. XDR (External Data Representation)

- A standard format for data representation
- Ensures data can be understood across different systems
- Used in heterogeneous environments
- Converts data into a machine-independent format
- Common in network communication systems

5. NDR (Network Data Representation)

- Used in distributed systems (like client-server models)
- Defines how data is formatted and transmitted over network
- Ensures systems with different architectures can communicate properly
- Used in technologies like RPC (Remote Procedure Call)

Presentation Layer Attacks

Since this layer deals with data formatting, compression and encryption, it is often targeted by attackers. Common attacks include:

- **Man-in-the-Middle (MITM) Attacks:** Interception of communication to steal sensitive data.
- **SSL/TLS Downgrade Attacks:** Forcing weaker encryption protocols.
- **Certificate Spoofing:** Using fake certificates to impersonate trusted entities.
- **Code Injection:** Exploiting vulnerabilities in data parsing or formatting.

Real-Life Examples

- **HTTPS websites** → encryption using SSL/TLS
- **File compression tools** → ZIP, RAR
- **Streaming platforms** → format conversion (video/audio)
- **Image formats** → JPEG, PNG conversion
- **Email attachments** → encoding and decoding